

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №15

по теме:

РЕАЛИЗАЦИЯ СИМВОЛЬНОЙ СИНХРОНИЗАЦИИ. ПРИМЕНЕНИЕ К
СЭМПЛАМ С ADALMPLUTO. РАЗРАБОТКА ГРАФИЧЕСКОГО
ИНТЕРФЕЙСА (GUI) ДЛЯ РАБОТЫ С SDR. ОКНА НАСТРОЕК SDR,
ВЫВОД РЕЗУЛЬТАТОВ ОБРАБОТКИ СЭМПЛОВ

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель	3
2	ПРАКТИЧЕСКАЯ ЧАСТЬ	4
2.1	Знакомство с ImGui и ImPlot	4
2.1.1	ImGui	4
2.1.2	ImPlot.....	5
2.1.3	Работа с ImGui (схематично)	7
2.1.4	Основные элементы ImGui и ImPlot	8
2.2	Подводный камень при работе с GUI	11
2.3	Результат	12
3	ВЫВОД	14

ВВЕДЕНИЕ

1.1 Введение

На этом занятии с помощью библиотек ImGui и ImPlot разработаем графический интерфейс на C++ для работы с SDR. Интерфейс должен отображать основные шаги обработки сигнала на стороне передатчика и приемника, настройки передатчика, приемника и SDR.

1.2 Цель

Познакомиться с библиотекой ImGui и ImPlot, реализовать GUI.

ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Знакомство с ImGui и ImPlot

2.1.1 ImGui

ImGui (Immediate Mode Graphical User Interface) — это библиотека и паттерн проектирования для создания быстрых графических интерфейсов (UI), где элементы отрисовываются каждый кадр непосредственно в коде. Она не хранит состояние UI, что упрощает синхронизацию и позволяет создавать инструменты для отладки, редакторы и интерфейсы для C++ приложений.

Приложение создает сцену из графических примитивов, таких как фигуры, линии и т.д., образуя сцены. Графическая библиотека хранит модель сцены в памяти. При отрисовки каждого кадра, приложение обращается к графическому API за обновлением сцены. Затем библиотека отрисовывает сцену.

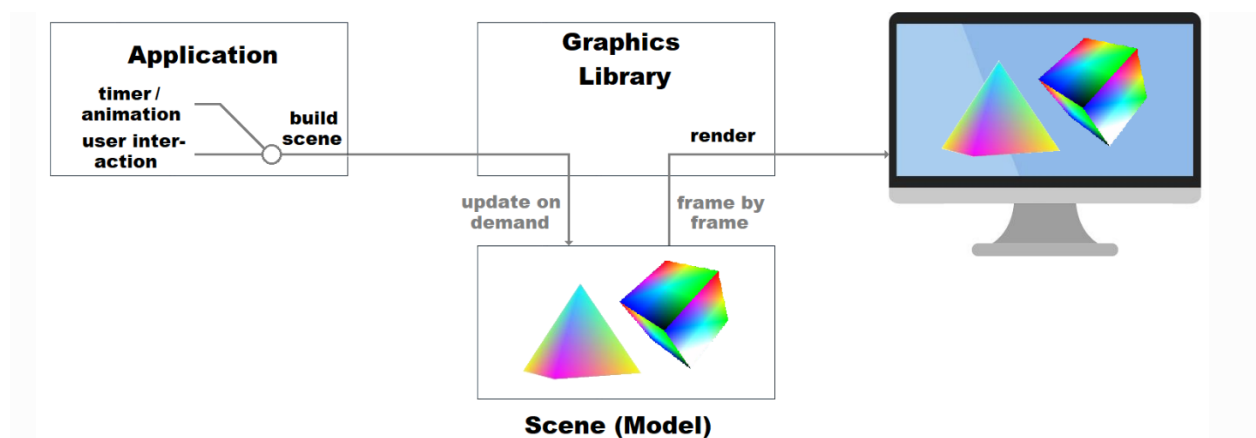


Рисунок 1 — Принцип работы ImGui

Особенности:

1. Интерфейс ImGui вызывается в цикле, каждая итерация цикла считается одним кадром.
2. Перед началом кадра библиотека должна получить все данные касательно положения мыши, нажатых клавиш и прочих вводных.

3. Затем вызывается функция, в которой пользователь библиотеки получает структуру `ui`, рисует свой интерфейс. В этот момент библиотека в реальном времени формирует все виджеты, для каждого виджета определяет попала ли на него мышь, и формирует вершины и треугольники, которые потом будут рисоваться.
4. Теперь библиотека возвращает все сформированные `mesh`'и и отдаёт их на рисование бэкенду. Он рисует это на экран.
5. И так повторяется 60 кадров в секунду (или больше).

2.1.2 ImPlot

ImPlot — это библиотека для быстрого построения 2D-графиков в реальном времени, разработанная специально для Dear ImGui. Она использует концепцию непосредственного выполнения (`immediate mode`), позволяя визуализировать данные с высокой производительностью, минимальными затратами ресурсов и легкой интеграцией без внешних зависимостей.



Рисунок 2 — Возможности ImPlot

2.1.3 Работа с ImGui (схематично)

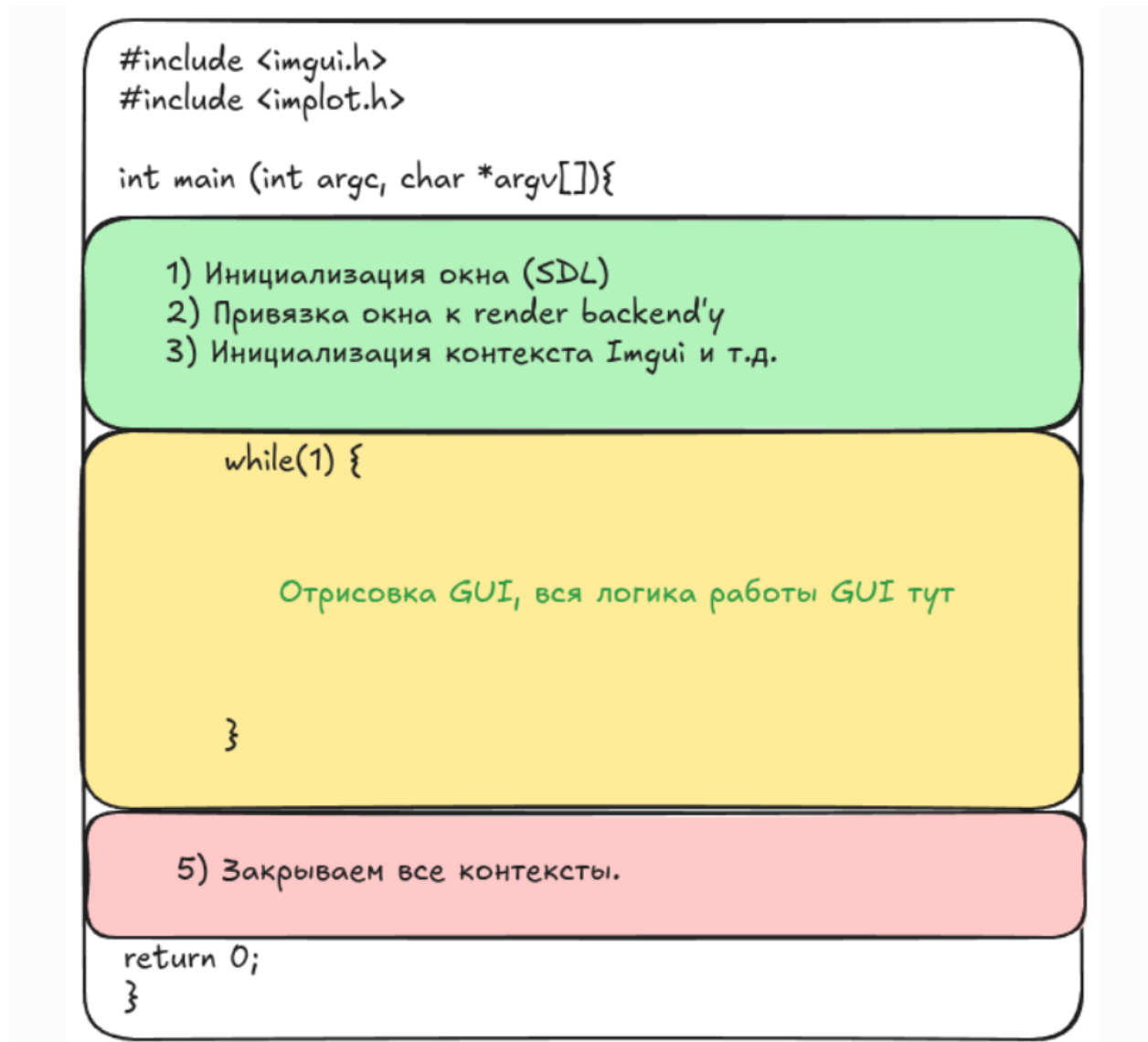


Рисунок 3 — Схема программы с ImGui

Пример инициализации:

```
// 1)          SDL
SDL_Init(SDL_INIT_VIDEO | SDL_INIT_TIMER);
SDL_Window* window = SDL_CreateWindow(
    "Backend start", SDL_WINDOWPOS_CENTERED,
    SDL_WINDOWPOS_CENTERED,
    1024, 768, SDL_WINDOW_OPENGL | SDL_WINDOW_RESIZABLE);
SDL_GLContext gl_context = SDL_GL_CreateContext(window);

// 2)          Dear ImGui
ImGui::CreateContext();
ImPlot::CreateContext();
```

```
//      \
ImGuiIO& io = ImGui::GetIO(); (void)io;
io.ConfigFlags |= ImGuiConfigFlags_DockingEnable;           //
                    Docking

// 2.1)      ImGui  SDL2  OpenGL backend '
ImGui_ImplSDL2_InitForOpenGL(window, gl_context);
ImGui_ImplOpenGL3_Init("#version 330");
```

2.1.4 Основные элементы ImGui и ImPlot

Input

Виджеты, предназначенные для ввода чисел (int, float, double)

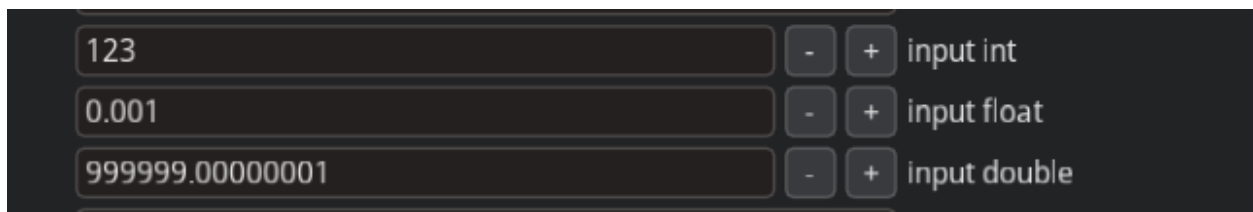


Рисунок 4 — Пример input виджетов

```
static int i0 = 123;
ImGui::InputInt("input int", &i0);

static float f0 = 0.001f;
ImGui::InputFloat("input float", &f0, 0.01f, 1.0f, "%.3f");

static double d0 = 999999.00000001;
ImGui::InputDouble("input double", &d0, 0.01f, 1.0f, "%.8f");
```

В функции передаются следующие аргументы:

1. "input double уникальный строковый идентификтор. Отображается возле виджета.
2. &d0 - адрес переменной, к которой будет привязан виджет (чье значение будет менять).
3. 0.01f - минимальное значение.
4. 1.0f - максисмальное значение.
5. "%.8f формат вывода.

Checkbox и radiobutton

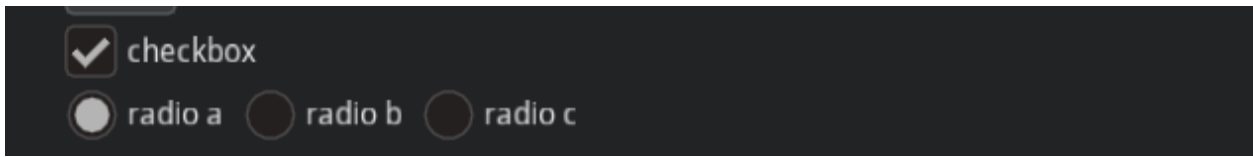


Рисунок 5 — Пример radiobutton и checkbox

```
static bool check = true;
ImGui::Checkbox("checkbox", &check);

static int e = 0;
ImGui::RadioButton("radio a", &e, 0);
ImGui::RadioButton("radio b", &e, 1);
ImGui::RadioButton("radio c", &e, 2);
```

Radiobutton третьим аргументом принимает значение, которое будет устанавливаться в переменной после нажатия на radiobutton.

Sliders

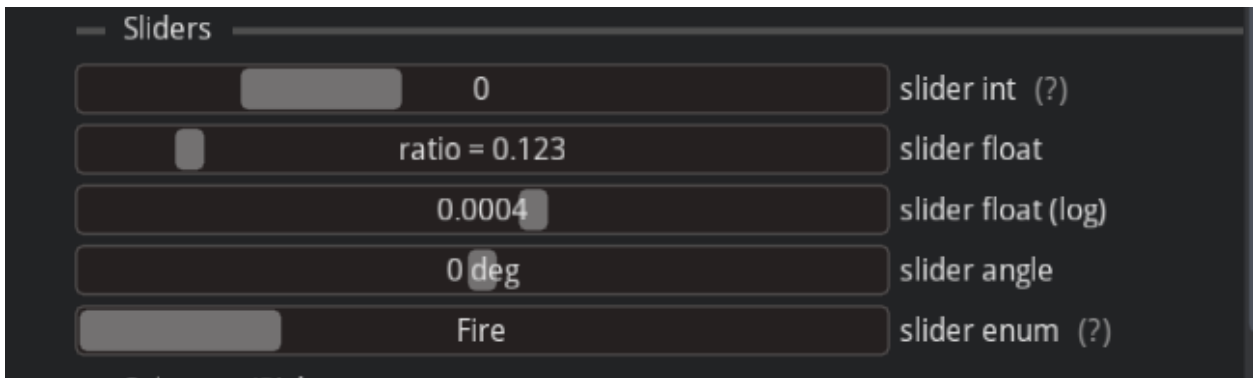


Рисунок 6 — Пример слайдеров

```
static int i1 = 0;
ImGui::SliderInt("slider int", &i1, -1, 3);

static float f1 = 0.123f, f2 = 0.0f;
ImGui::SliderFloat("slider float", &f1, 0.0f, 1.0f, "ratio = %.3f");
ImGui::SliderFloat("slider float (log)", &f2, -10.0f, 10.0f, "%.4f", ImGuiSliderFlags_Logarithmic);
```

Синтаксис очень похож на виджеты Input, но есть возможность установить флаги, например, для логарифмической шкалы.

Plots

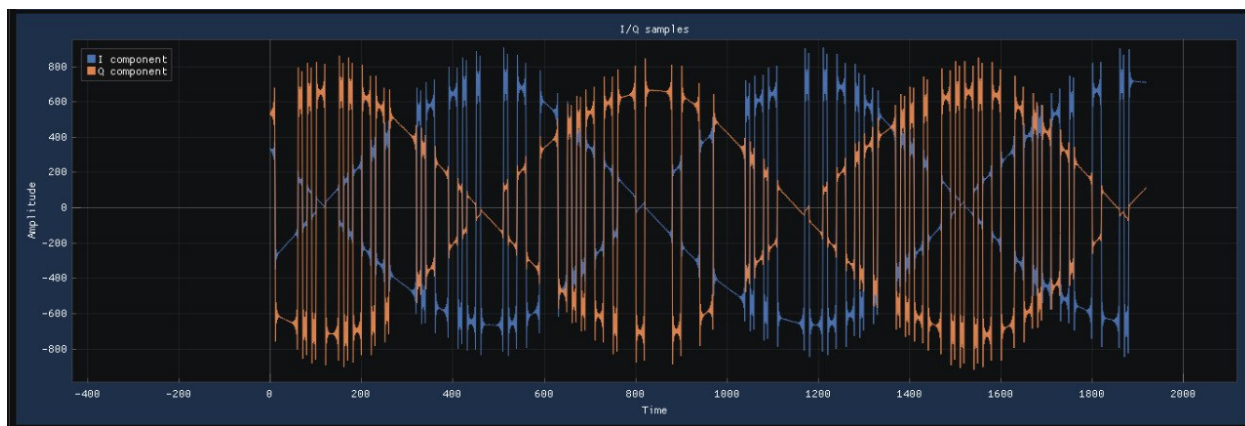


Рисунок 7 — Пример графика

Код:

```
if (ImPlot::BeginPlot("I/Q samples", ImVec2(-1,
plot_height)))
{
    ImPlot::SetupAxes("Time", "Amplitude");
    ImPlot::PlotLineG("I component", get_I<int16_t>,
        &rx_config.rx_samples,
        rx_config.rx_samples.size());
    ImPlot::PlotLineG("Q component", get_Q<int16_t>,
        &rx_config.rx_samples,
        rx_config.rx_samples.size());
    ImPlot::EndPlot();
}
```

Вызвать само окно под график можно с помощью `ImPlot::BeginPlot()`, куда необходимо передать название графика (это в то же время его уникальный ID), размер графика в виде структуры `ImVec2(width, height)`. -1 означает, что занимает вся доступная ширина. Вызов можно удобно делать в `if`, что предотвращает появление ошибок.

Далее идет более детальная настройка графика. С помощью `ImPlot::SetupAxes()` настраиваются подписи для осей. `ImPlot::PlotLineG()` позволяет отобразить

данные. Функция принимает название выводимых данных, сами данные (адрес), callback функцию и размер данных. Callback функция здесь нужна по той причине, что ImPlot умеет работать только с дефолтными массивами и только с дефолтными типами данных. У меня, например, была задача вывести вектор комплексных чисел, а это значит, что вектор надо было распарсить на 2 обычных массива с мнимой и реальной частью. Делать это крайне неудобно, поэтому разработчики ImPlot позаботились об этом заранее и дали возможность передать в функцию указатель на callback функцию, которая будет автоматически применяться к каждому выводимому объекту.

По большому счету пользователю требуется только описать то, как парсить данные, а всю остальную рутинную работу сделает ImPlot.

Callback функция пишется в строго определенном стиле:

```
template <typename T>
ImPlotPoint get_I(int idx, void *user_data) {
    auto *vec = static_cast<std::vector<std::complex<T>>*>(user_data);
    return ImPlotPoint(idx, (*vec)[idx].real());
}
```

Функция должна обязательно иметь сигнатуру, которая указана выше. Здесь объект ImPlotPoint - точка на графике, idx - индекс, который ImPlot будет подавать на функцию, user_data - данные, которые будет подавать ImPlot. Логика такая: ImPlot, допустим, хочет вывести вектор, он берет нулевой элемент вектора и индекс (0) подставляет вместо idx, а сам элемент вместо user_data. Дальше идет обработка, в ходе которой должен быть сформирован и возвращен объект точки, который ImPlot отрисует.

2.2 Подводный камень при работе с GUI

При работе с GUI обязательным является вынос логики работы GUI в отдельный поток, поскольку задачи основного потока могут тормозить работу GUI и наоборот. Запустить новый поток несложно, но встает вопрос обмена данными между потоками. Для этого использовались обычные структуры, которые по ссылке передавались в потоки. Таким образом, например, основной поток

принимал с SDR семплы, а поток GUI сразу же читал их и выводил, причем в такой схеме не требуется синхронизировать потоки, потому что GUI только читает.

```
std::thread gui_thread(run_gui, std::ref(tx_config),
                      std::ref(rx_config),
                      std::ref(sdr_config));

std::thread sdr_thread(SDR_run, std::ref(sdr_config),
                      std::ref(tx_config),
                      std::ref(rx_config));

gui_thread.join();
sdr_thread.join();
```

2.3 Результат

Имея небольшой функционал, описанный выше, можно построить вполне неплохой GUI:

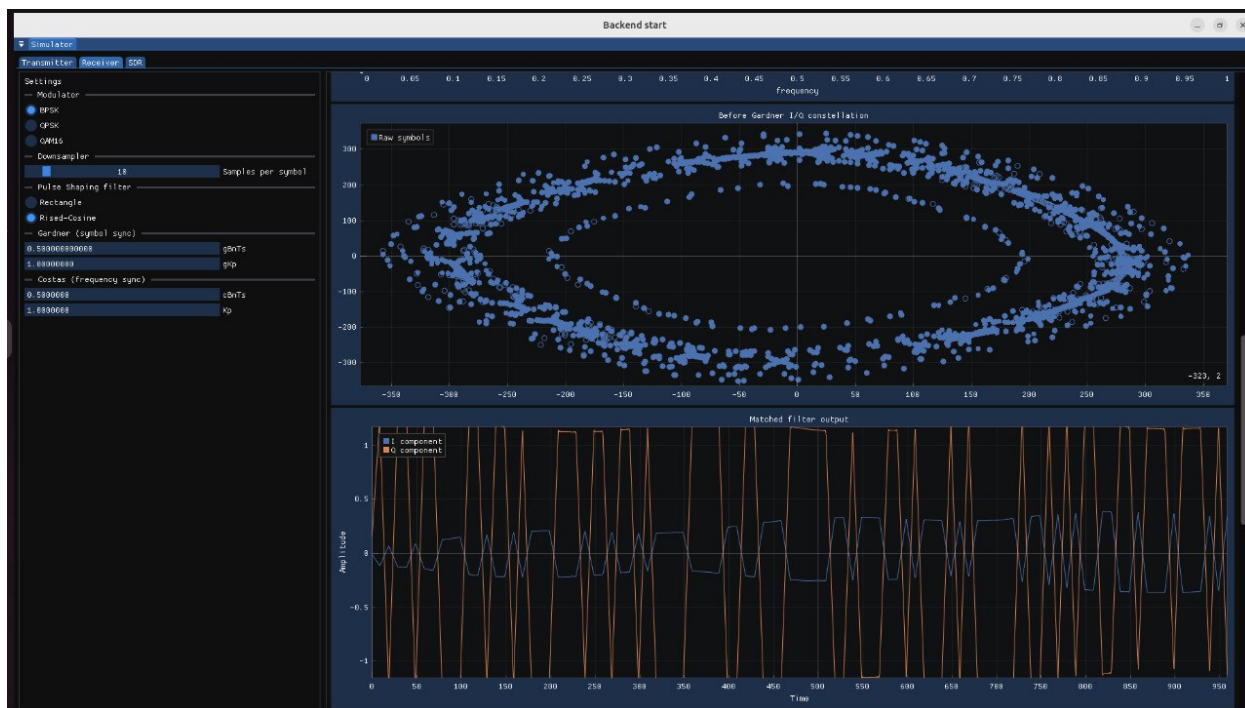


Рисунок 8 — GUI для RX

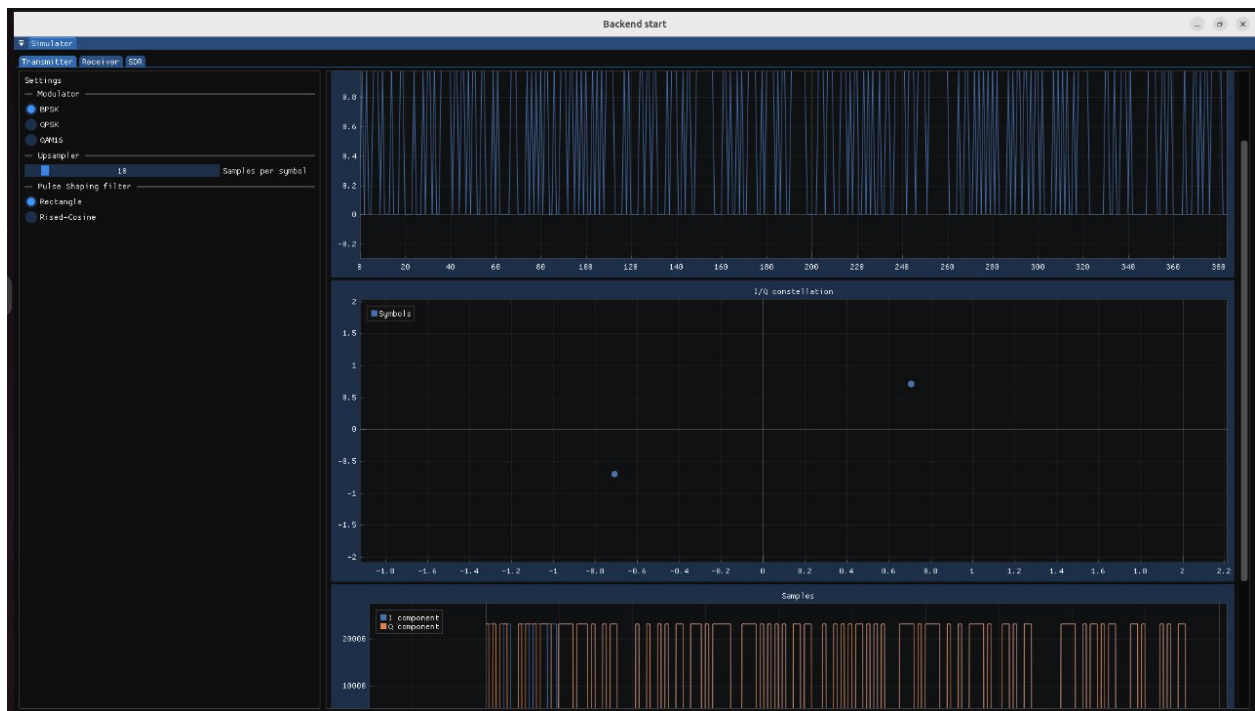


Рисунок 9 — GUI для TX

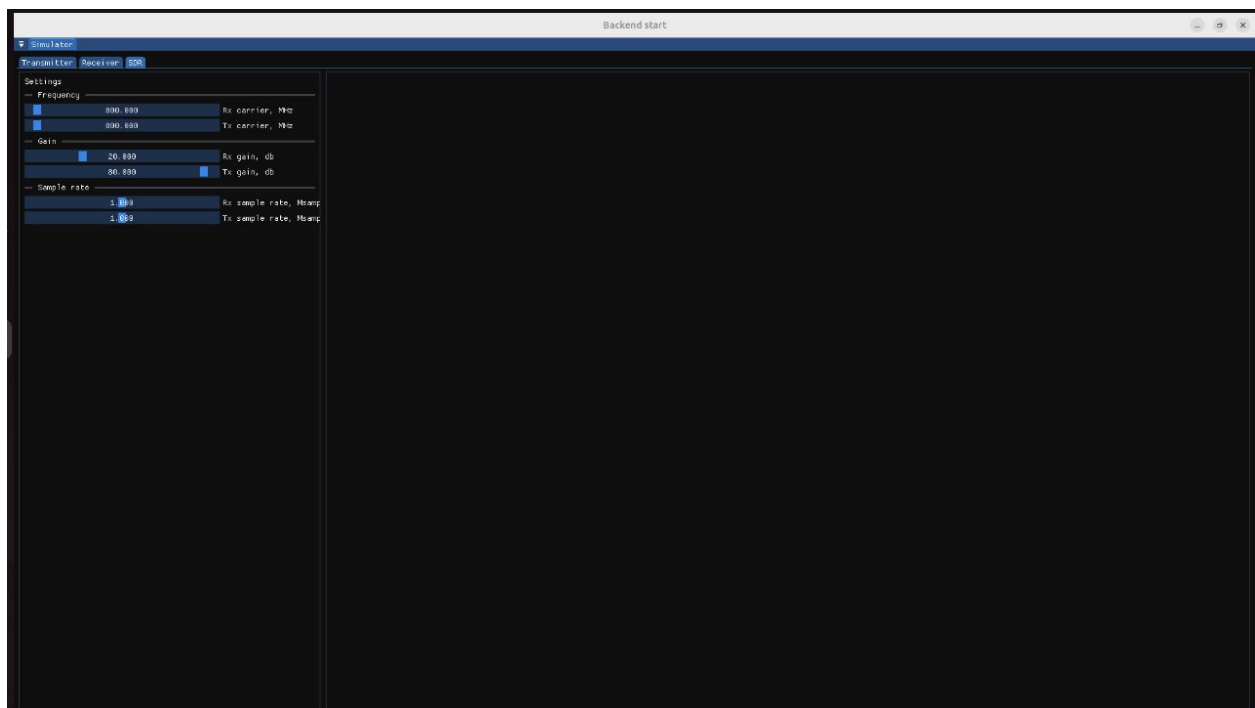


Рисунок 10 — GUI для SDR

ВЫВОД

В ходе занятия я познакомился с библиотеками ImGui и ImPlot и с их помощью написал графический интерфейс на C++, работающий в режиме реального времени и позволяющий менять параметры передатчика, приемника и SDR, а также наблюдать за процессом цифровой обработки сигналов.